

VÙNG 4: KIẾN TRÚC GIAO TIẾP HỆ THỐNG & C-ABI INTEROP

Hợp đồng nhị phân tầng C-ABI giữa F# NativeAOT Core và Fcitr5 C++ Adapter

Mục lục tài liệu

1. Tổng quan Kiến trúc Đa tầng (Tiered System Architecture)	1
2. Đặc tả Hợp đồng C-ABI (C-ABI Contract Definition)	1
2.1. Cấu trúc Sự kiện Đầu vào (Native Key Event)	1
2.2. Cấu trúc Chỉ thị Phản hồi (Native Output Action)	2
3. Quản lý Bộ nhớ Phi Quản lý & Vòng đời Thể hiện (Memory & Instance Lifecycle)	2
3.1. Bảng Hàm Xuất Bản Chuẩn C (Exported C Functions)	2
3.2. Nguyên tắc An toàn Bộ nhớ (Memory Safety Rules)	3

1. Tổng quan Kiến trúc Đa tầng (Tiered System Architecture)

Hệ thống BambooMintKey được tách biệt thành hai phân vùng độc lập qua ranh giới nhị phân (Binary Boundary):

1. Core Engine (F# NativeAOT):

- Chịu trách nhiệm toàn bộ logic xử lý ngôn ngữ, DFA Parser, Phonotactics và bộ đệm trạng thái bất biến.
- Được biên dịch tĩnh thành thư viện liên kết động (libbmk_core.so trên Linux hoặc bmk_core.dll trên Windows) mà không cần cài đặt .NET Runtime.
- Không chứa bất kỳ phụ thuộc nào liên quan đến UI hay khung nhập liệu cụ thể.

2. Host Adapter (Fcitr5 C++ Addon):

- Cài đặt giao diện fcitr5::InputMethodEngineV2 và fcitr5::Factory.
- Đón nhận các sự kiện phím vật lý từ X11/Wayland thông qua Fcitr5 Event Loop.
- Chuyển tiếp mã phím qua ranh giới C-ABI và nhận lại các chỉ thị hành động (Commit, Replace, Forward).

Sơ đồ Phân tầng Dữ liệu

[X11 / Wayland KeyEvent] → [Fcitr5 C++ Addon] → (C-ABI Boundary) → [F# NativeAOT Core]
 [Client App / Text Box] ← [Commit / Forward] ← (C-ABI Boundary) ← [Engine Action Result]

2. Đặc tả Hợp đồng C-ABI (C-ABI Contract Definition)

Để đảm bảo khả năng tương thích nhị phân tuyệt đối, dữ liệu trao đổi qua ranh giới FFI (Foreign Function Interface) chỉ sử dụng các kiểu dữ liệu nguyên thủy (Primitives) và cấu trúc bộ nhớ phẳng có thứ tự tường minh (Sequential Layout).

2.1. Cấu trúc Sự kiện Đầu vào (Native Key Event)

Mỗi phím bấm gửi từ C++ sang F# Core được đóng gói trong một struct 64-bit có cấu trúc cố định:

$$\text{NativeKeyEvent} = (k_{\text{code}}, c_{\text{unicode}}, m_{\text{flags}})$$

Trong đó:

- $k_{\text{code}} \in \text{uint32}$: Mã phím ảo của hệ thống (X11 Keysym hoặc Linux Evdev Scancode).
- $c_{\text{unicode}} \in \text{uint32}$: Mã UTF-32 của ký tự sinh ra (hoặc 0 nếu là phím chức năng).
- $m_{\text{flags}} \in \text{uint32}$: Mặt nạ bit chứa trạng thái của Shift, Ctrl, Alt, CapsLock, Super.

2.2. Cấu trúc Chỉ thị Phản hồi (Native Output Action)

Sau khi xử lý, F# Core trả về một bản ghi chỉ thị hành động gồm 4 trường dữ liệu:

Tên trường	Kiểu dữ liệu	Mô tả nghiệp vụ
action_type	int32	Mã định danh loại hành động: 0 (Pass), 1 (Consume), 2 (Commit), 3 (Replace).
backspaces	int32	Số lượng phím Backspace mà Host cần mô phỏng trước khi gửi chuỗi mới.
output_text	const char*	Con trỏ chuỗi byte UTF-8 kết quả (kết thúc bằng ký tự null).
text_length	int32	Độ dài chuỗi byte UTF-8 (không tính null terminator).

Bảng 1: Bố cục bộ nhớ của Native Output Action

3. Quản lý Bộ nhớ Phi Quản lý & Vòng đời Thể hiện (Memory & Instance Lifecycle)

Kiến trúc F# NativeAOT sử dụng mô hình Handle Opaque Pointer để quản lý trạng thái đa thể hiện (Multi-instance), cho phép mỗi cửa sổ ứng dụng hoặc ngữ cảnh nhập liệu sở hữu một máy trạng thái độc lập.

3.1. Bảng Hàm Xuất Bản Chuẩn C (Exported C Functions)

Bộ thư viện lõi xuất bản 4 hàm ngoại vi chính thức qua cơ chế [<UnmanagedCallersOnly>]:

1. Khởi tạo Thể hiện (Allocate):

`bm_k_engine_create() : nativeint`

Cấp phát một phiên bản EngineState mới trên bộ nhớ heap và trả về con trỏ mờ (`IntPtr / void*`).

2. Hủy bỏ Thể hiện (Free):

`bm_k_engine_destroy(handle : nativeint) : void`

Giải phóng bộ đệm và thu hồi tài nguyên gắn liền với handle đã cấp phát.

3. Xử lý Sự kiện Phím (Process):

`bm_k_engine_process(handle : nativeint, event : NativeKeyEvent *, result : NativeActionResult *) : int32`

Nạp sự kiện phím vào máy trạng thái, ghi đè trực tiếp kết quả vào con trỏ struct do phía C++ cung cấp.

4. Thiết lập Cấu hình (Configure):

`bm_k_engine_set_option(handle : nativeint, option_id : int32, val : int32) : void`

Thay đổi kiểu gõ (Telex/VNI), kiểu đặt dấu (Mới/Cũ), hoặc kích hoạt kiểm tra chính tả khi đang chạy.

3.2. Nguyên tắc An toàn Bộ nhớ (Memory Safety Rules)

Để triệt tiêu hiện tượng rò rỉ bộ nhớ (Memory Leak) và con trỏ treo (Dangling Pointer):

- **Quy tắc cấp phát:** Phía C++ Adapter luôn là bên chủ động cấp phát bộ đệm tiếp nhận kết quả (NativeActionResult) trên ngăn xếp (Stack) trước khi truyền địa chỉ con trỏ sang F# Core.
- **Quy tắc chuỗi ký tự:** Con trỏ `output_text` trỏ tới vùng đệm nội bộ bất biến của Engine. Phía C++ Adapter phải sao chép dữ liệu ngay lập tức vào chuỗi `std::string` và không bao giờ được gọi lệnh giải phóng (`free / delete`) đối với con trỏ này.